Criterion A

Description of Scenario

The Black Knights robotics team's business team needs a better way of organizing donations. Right now, the team doesn't know which companies have been contacted, which companies to follow up on, and which companies have donated. The donations from previous years are also disorganized. The solution that is being developed is a database that can store donations, the status of a fundraising opportunity, and different company's contact information. The client is my dad, who mentors the business team. The choice of my client is appropriate because he can test the program to see if it would work and show it to the business team members and give me feedback. He also knows what methods for organizing information are effective since he has seen how the business team used to organize donations. This product is being developed to help the business team members easily keep track of donations, donors, and which company they're assigned to. They can also search for donation history of different companies, look up companies' contact information, and check the status of fundraising opportunities.

Rationale for the Proposed Product

I will create a website that connects to a web database because my client wants the business team to be able to access it without special software since not all of the business team members will be able to download something. The program will use a database because it is easier to store data that way than creating lots of variables. The data will be displayed in a table which will make it easier for the team member to read. The website will only have a few boxes to type things into, so it will be easier for the business team member to figure out how to input information. As seen in Appendix 1, the website will keep track of current and previous fundraising opportunities. The team member will be able to add contact information for donors, fundraising opportunities and their progress, and create an account. The website will also have a way for the business team member to search the database for something like donations made by a certain company. That will be helpful because the client wants to be able to search for things such as donation history and donor information. If I have extra time, the client would also like the website to send email or text message reminders to team members.

I will use 000webhost to host my website and a combination of PHP, HTML, and CSS. My rationale for using 000webhost is because it is free and it supports databases and PHP. HTML and CSS are commonly used for creating websites. I decided to use PHP since I would not need to download anything to write the code since I could edit documents through 000webhost. Also, if any of the business team members need to modify the product in the future, they will be able to sign in using the team's Google account to change the files.

Success Criteria for the Product

- Ability to login
- Track the progress of a fundraising opportunity
- Track basic information of a fundraising opportunity like contact information
- Easy to input information and edit information
- Be able to search for different entries
- See the progress of any fundraising opportunity
- See the progress of fundraising opportunities assigned to a specific member

- See the donation history of a donor

Word Count: 488

Criterion B: Record of Tasks

Task Number	Planned Action	Planned Outcome	Time Estimated	Target Completion Date	Criterion
1	Find a client	Figure out who I will be making my project for.	1 day	1/25/2020	A
2	Meet with my client to discuss the problem and success criteria	Understand what the problem is and the success criteria	1 day	1/26/2020	A
3	Think of a solution for the product and propose it to the client	Have a good idea of what the solution will look like	1 day	1/27/2020	A
4	Write Criterion A	Finished rough draft of Criterion A	1 day	1/28/2020	A
5	Make solution design diagrams	Have completed solution design diagrams	2 days	2/3/2020	B
6	Make a testing plan	Have a completed testing plan	1 day	2/4/2020	B
7	Find a host for my website	Have a host that can host my website	1 day	2/6/2020	A
8	Learn HTML, CSS, and PHP	Know basic programming for HTML, CSS, and PHP	7 days	2/17/2020	C
9	Create a database and tables and use PHP to connect to the database	Have PHP code successfully connect to the database and complete the tables in the database	2 days	2/19/2020	C
10	Create a style sheet	Completed style sheet	2 days	2/21/2020	
11	Create documents for all the pages for the website and a navigation menu	Completed blank pages with a completed navigation menu	2 days	2/25/2020	C
12	Create the home page	Completed home page	1 day	2/25/2020	C
13	Create the login, log out	User can create an	5 days	3/3/2020	C

	and create account page	account, log in, and log out			
14	Make some pages only accessible after a user has logged in	User can only access certain pages after logged in	1 day	3/4/2020	C
15	Create an Add Donor page	User can add a donor to the database	1 day	3/5/2020	C
16	Create an Enter Data page	User can add a fundraising opportunity to the database	1 day	3/9/2020	C
17	Meeting with client to get feedback on login system, add donor, and enter data	Know what might need to be changed and what works well	1 day	3/9/2020	C
18	Create a search and select feature for the Edit Data page	User can search for an entry to edit and select it	4 days	3/13/2020	C
19	Display current data	The user can see the current data for an entry when they edit data, so they know what to edit. They only need to edit what needs to be changed instead of retyping everything.	2 days	3/17/2020	C
20	Create feature to edit the data	User can edit data	2 day	3/22/2020	C
21	Add page to search for entries and donors	User can search for entries/donors by selecting what they are searching by or display all entries/donors	3 days	3/27/2020	C
22	Meeting with client to receive feedback on changes to enter data page and show the edit data and search pages	Verify that the enter data page is good and know what needs to be changed for the edit data and search pages	1 day	3/27/2020	C
23	Create dashboard	User can see the	2 days	3/29/2020	C

		progress for their assigned tasks and what their next steps are			
24	Meeting with client to get feedback on dashboard, edit data, and search pages	Know what needs to be changed before the final meeting	1 day	3/29/2020	C
25	Add dropdown menus for progress, next step, and year in the enter data form	The user can select an option from the dropdown menus instead of creating their own.	2 days	3/31/2020	C
26	Try figure out how to add dropdown menus for progress, next step, and year in the edit data form	The dropdown menu for the edit data page displays the currently selected option. The currently selected option isn't displayed again after clicking on the dropdown menu. See Appendix 2.	1 day	4/1/2020	C
27	Final meeting with client	Verify that the project meets the success criteria	1 day	4/1/2020	C
28	Add dropdown menus for progress, next step, and year in the edit data form	The dropdown menu for the edit data page displays the currently selected option. The currently selected option isn't displayed again after clicking on the dropdown menu.	1 day	4/2/2020	C
29	Test product	The product works			
30	Write Criterion C	Completed Criterion C	1 day	4/3/2020	C
31	Make video for Criterion D	Completed Criterion D	1 day	4/3/2020	D
32	Write Criterion E	Completed Criterion E	1 day	4/3/2020	E
33	Revise Criterion A	Revised Criterion A	1 day	11/12/2020	A

34	Revise Criterion B	Revised Criterion B	1 day	12/14/2020	B
35	Add more complicated features	Added more complicated features	1 day	3/29/2021	C
36	Revise Criterion C	Revised Criterion C	1 day	3/29/2021	C
37	Revise Criterion D	Revised Criterion D	1 day	3/29/2021	D

Criterion B: Design Overview

Database Tables

Students

Column Name	Data Type	Description	Other Information
Username	Varchar	Username	Primary key
Password	Varchar	Password	Password is hashed
First	Varchar	First name	
Last	Varchar	Last name	
Email	Varchar	Email address	

Donor

Column Name	Data Type	Description	Other Information
Company	Varchar	Company name	Primary key
Contact	Varchar	Contact name	
Phone	Number	Phone number	
Email	Varchar	Email address	
Address	Varchar	Physical address	
City	Varchar	City	

Fundraising

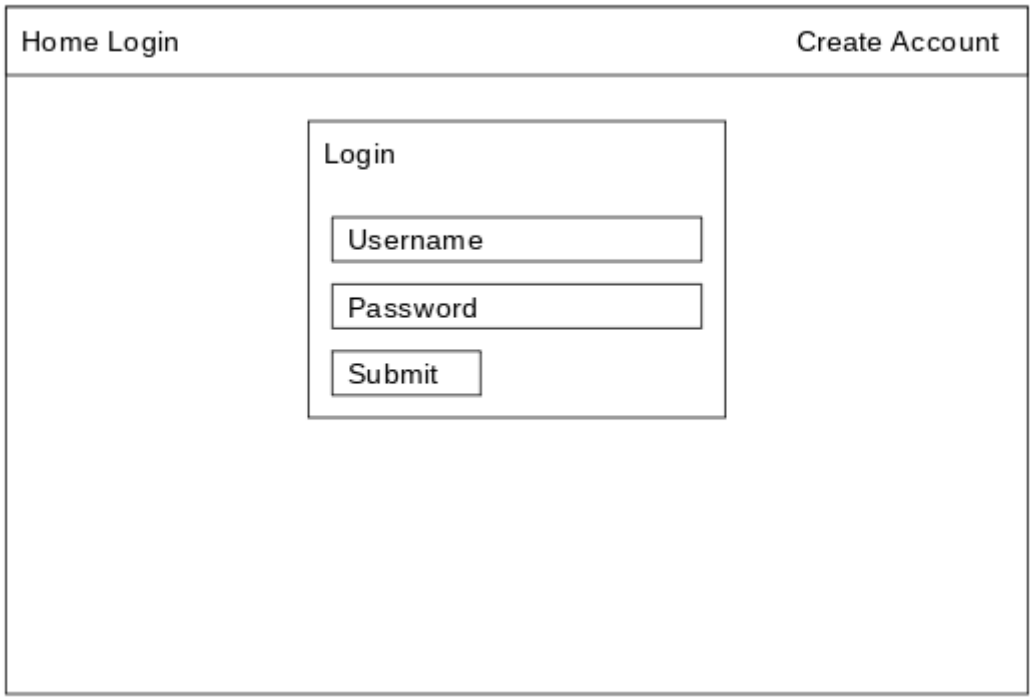
Column Name	Data Type	Description	Other Information
Company	Varchar	Company name	
Amount	Number	Amount of money targeted	Default: Null
Success	Number	Probability of success	Default: Null
Progress	Varchar	Progress of donation	
Next Step	Varchar	Next step for donation	
Year	Number	Year donation was made	Default: Null

Member_First	Varchar	Student assigned first name	Default: Null
Member_Last	Varchar	Student assigned last name	Default: Null

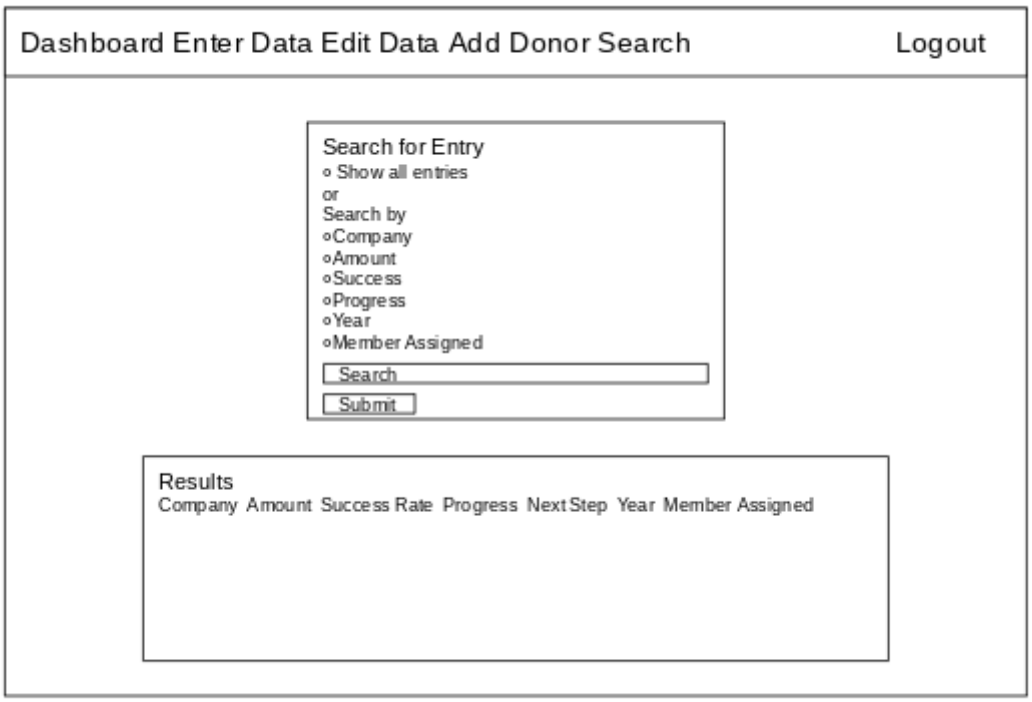
Forms

Form Name	**Outline**
create_account	Lets the user create an account
login	Lets the user log in
enter_data	Lets the user add a new fundraising opportunity
search_data	Lets the user search for an entry to edit
edit_data	Lets the user select which entry from the search results to edit
edit	Lets the user edit the entry
add_donor	Lets the user add a donor to the database
search_donation	Lets the user search the Fundraising table
search_donor	Lets the user search the Donors table

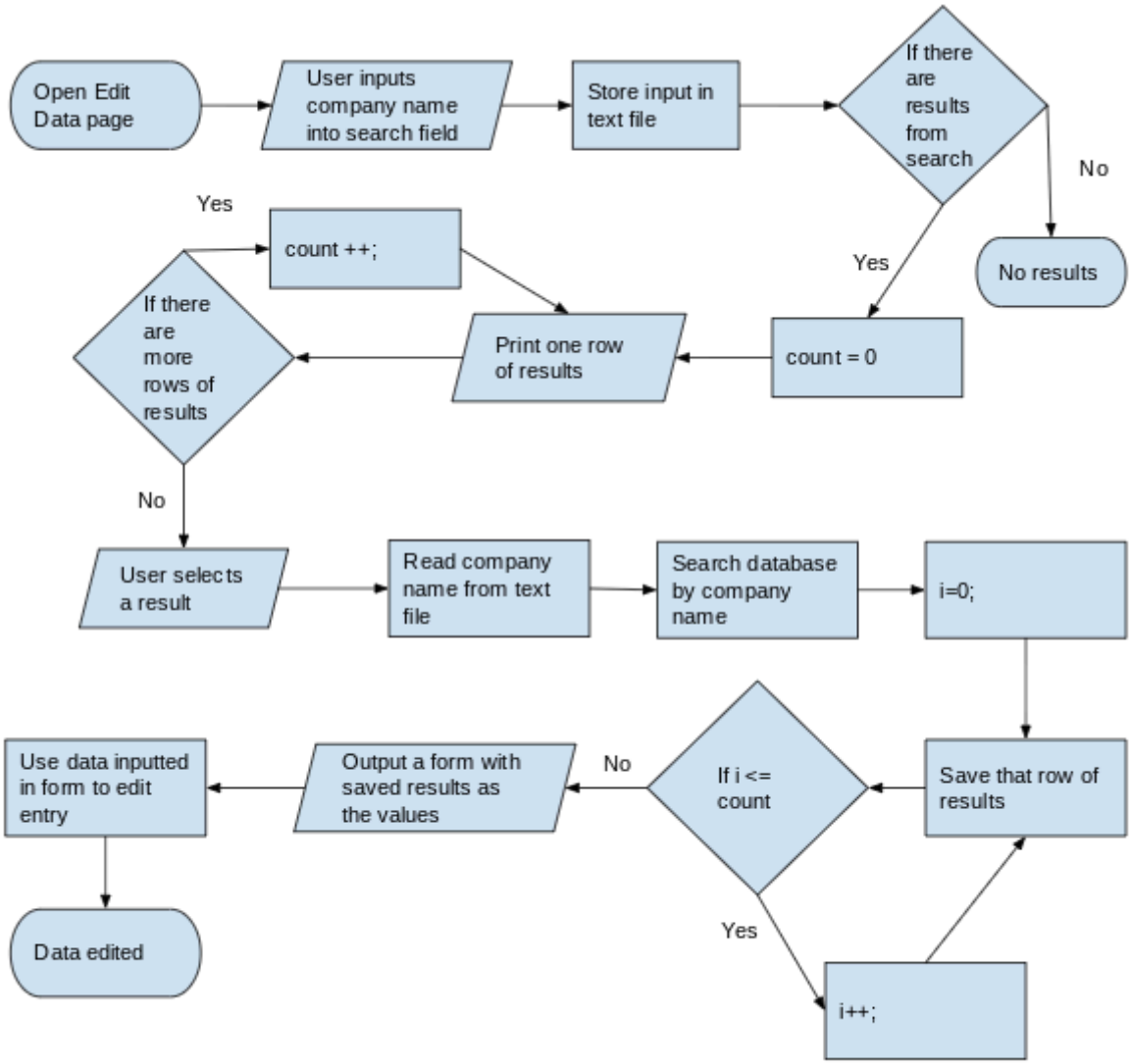
Screen Sketch of Login Page



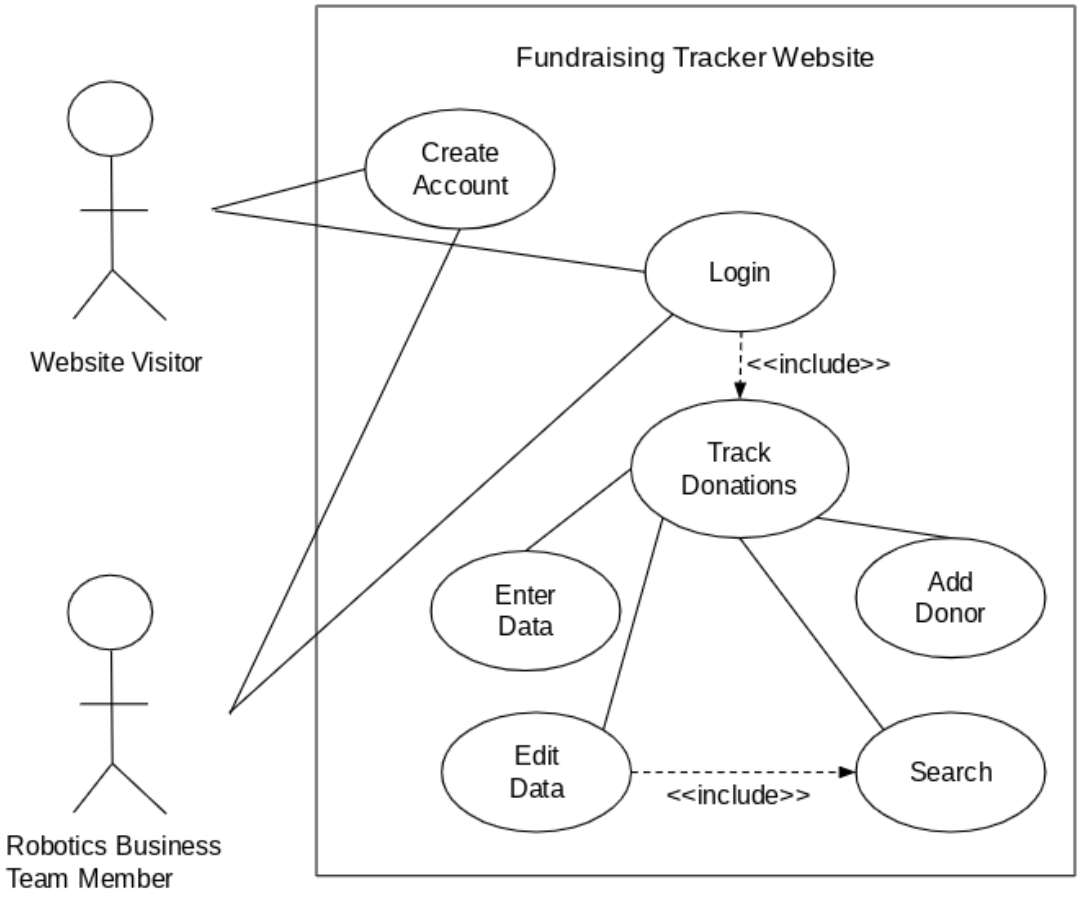
Screen Sketch of Search Page (after logging in)



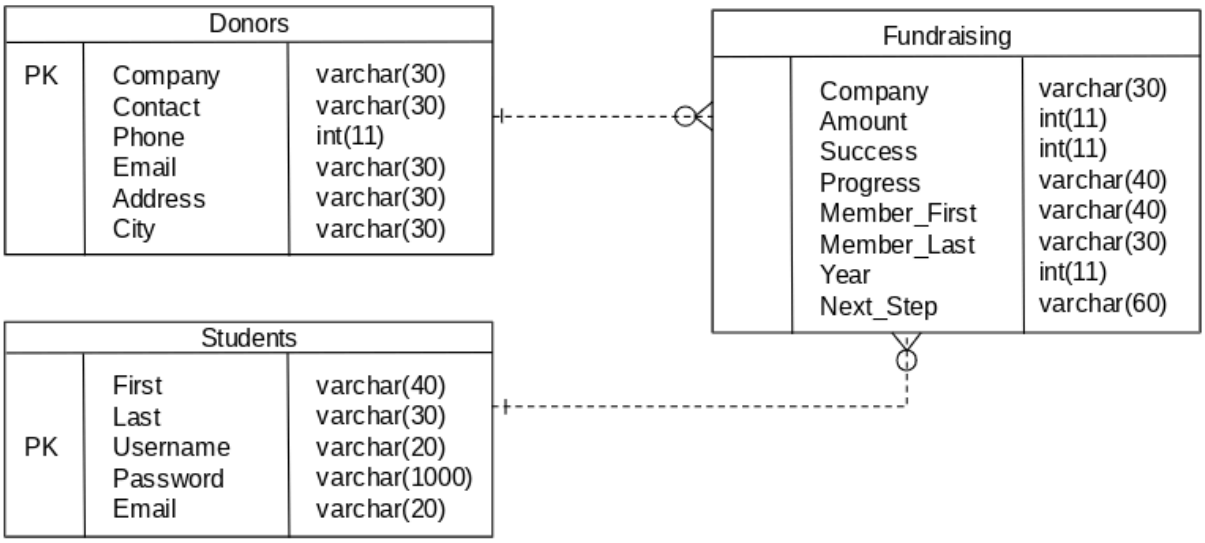
Flow Chart of Edit Data Page



Use Case Diagram for Fundraising Tracker



Entity Relationship Diagram for Database



Criterion B: Testing Plan

What is Being Tested	Method for Testing	Example
Create account and login forms work correctly	Test confirm password field and make sure the login form only works with the correct password. Test that the password field works with any type of character.	Try typing a different password when confirming it and see if the account can still be created. Try logging in with incorrect passwords to see if the program lets the user log in.
Tracks the progress of a fundraising opportunity	Entering a fundraising opportunity into the database and seeing if it stores the progress. Also make sure that it doesn't work if nothing is selected.	Selecting "Qualify Target" and entering it should store "Qualify Target" in the database. It can be checked by going to the search page and searching for that entry. In the progress column, it should say "Qualify Target."
Stores contact information of a donor	Enter contact information for a donor into the database and see if it stores the information. Make sure the donor still gets stored if no contact information is entered.	Enter information into the Add Donor page. Go to the search page and search for that donor. All the information that was stored should be correct.
Searches for different entries	Test by searching for all the entries and searching by company name, amount, success rate, progress, year, and member name. Check that typing words or negative numbers in the amount field doesn't work.	Selecting "Show all Entries" should display all the entries. Searching by specific columns should display all the entries that match the search.
See the progress of any fundraising opportunity	Selecting "Show all Entries" should display the results which have a progress column where the user can see the progress.	
See the progress of fundraising opportunities assigned to a specific member	Selecting to search by member name and searching should display a table with a column labelled progress. Or if the member is the person who is logged in, they should be able to go to their dashboard and see the progress of all the	

	fundraising opportunities they're assigned to.	
Edit data page works correctly	The edit data page lets the user select the data that they want to edit and displays it correctly in the editing form. The dropdown menus select the right values and don't have duplicate options. Check that the correct information was edited.	When the user selects an entry to edit, the page displays all the right information. If the year is 2020, the dropdown menu should show 2020 as the default value. When the user clicks on the dropdown menu, it shouldn't display 2020, 2019, 2020, 2021. It should display 2020, 2019, 2021 instead.

Criterion C: Development

Techniques used in developing the product:

1. Database tables
2. Using PHP to print HTML code
3. Dropdown menus with a preselected option
4. Edit Data page (Algorithmic Thinking)
5. Reading and writing text files
6. Downloading HTML tables as CSV files
7. Sending reminder emails

1. Database Tables

An important component of this product is database tables since the client wanted the product to store information about students, donors, and fundraising opportunities. A database also makes it easier to get stored data and print it in an HTML table using arrays. The database tables were created using one of the tools available with the 000webhost web host, phpMyAdmin. The code used to connect to the database was a modified version of code from an example from w3schools.com (PHP MySQL Connect to database).

Connecting to the database:

```php
<?php
    session_start();
    if(!isset($_SESSION['username'])){
       header("Location:login.php");
    }

    try{
        $servername = "localhost";
        $username = "id12728702_fairviewrobotics";
        $password = "blackknights";
        $dbname="id12728702_fundraising";

        $conn = new PDO("mysql:host=$servername;dbname=$dbname", $username, $password);

        $conn->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
    }
    catch(PDOException $e){
        echo "Connection failed.";
    }
?>
```

Adding data to the database:

```php
if(isset($_POST['add_donor'])){
    $company = !empty($_POST['company']) ? trim($_POST['company']) : null;
    $contact = !empty($_POST['contact']) ? trim($_POST['contact']) : null;
    $phone = !empty($_POST['phone']) ? trim($_POST['phone']) : null;
    $email = !empty($_POST['email']) ? trim($_POST['email']) : null;
    $address = !empty($_POST['address']) ? trim($_POST['address']) : null;
    $city = !empty($_POST['city']) ? trim($_POST['city']) : null;

    $sql = "INSERT INTO Donors (Company, Contact, Phone, Email, Address, City) VALUES
        (:company, :contact, :phone, :email, :address, :city)";
    $stmt = $conn->prepare($sql);

    $stmt->bindvalue(':company', $company);
    $stmt->bindvalue(':contact', $contact);
    $stmt->bindvalue(':phone', $phone);
    $stmt->bindvalue(':email', $email);
    $stmt->bindvalue(':address', $address);
    $stmt->bindvalue(':city', $city);

    try {
        $stmt->execute();
        echo "The donor was successfully added.";
    }
    catch (PDOException $e){
        echo "Error: " . $e->getMessage();
    }
}
```

Donors table:

Company	Contact	Phone	Email	Address	City
Ball	Contact Name	0	contactemail@ball.com	Address	Boulder

Students table:

First	Last	Username	Password	Email
Kiana	Fan	kjfan01	$2y$12$skQGXUPT69bdCVkfHnyKEeNy4FBYAp8xb9s2waOD0H3...	kjfan01@bvsd.org

Fundraising table:

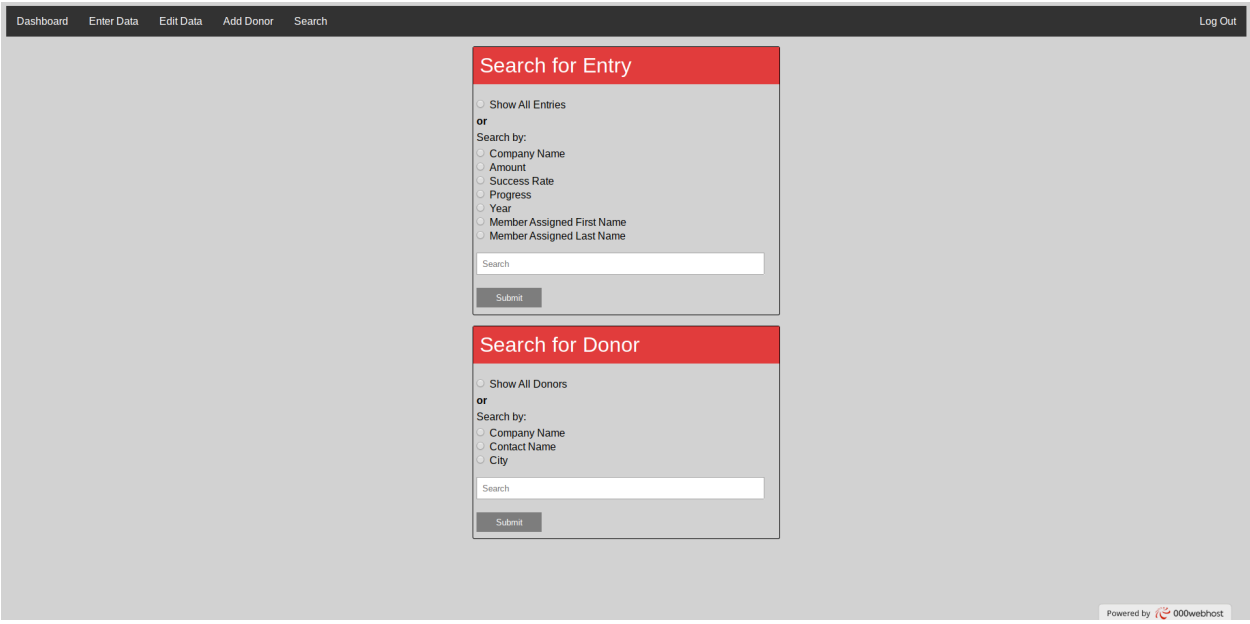
Company	Amount	Success	Progress	Member_First	Member_Last	Year	Next_Step
Google	NULL	50	Initial Contact	NULL	NULL	2020	Visit Contact in Person
Cosmos	NULL	70	Initial Contact	NULL	NULL	2020	Wait for Contact to Respond
Medtronic	5000	100	Closed Won	NULL	NULL	2019	NULL
Wong Orthodontics	200	NULL	Closed Won	NULL	NULL	2019	NULL
Ball	500	100	Closed Won	NULL	NULL	2019	NULL
Premier Members Credit Union	500	100	Closed Won	NULL	NULL	2020	NULL
McGuckins	50	100	Closed Won	NULL	NULL	2020	NULL

The user can enter and search for data from those tables on the Add Donor, Enter Data, Edit Data, and Search pages.

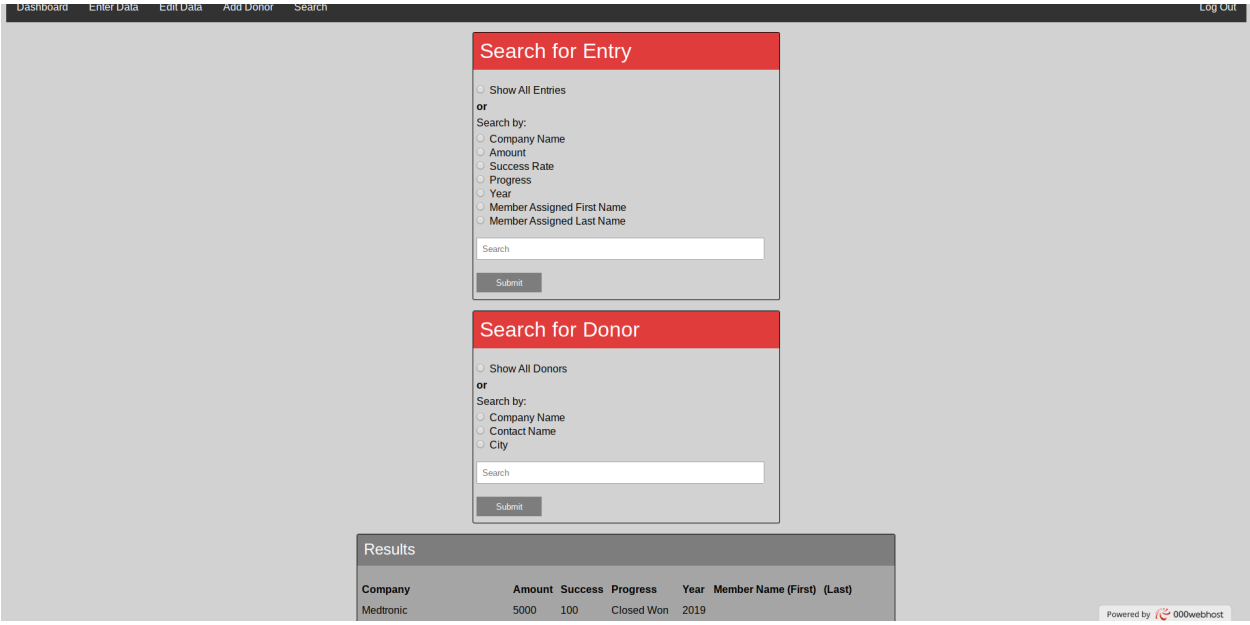
2. Using PHP to Print HTML

Another component is using PHP to print HTML. In pages like the Edit Data and Search pages, some parts of the website were only supposed to appear after the user submitted a form. In the Edit Data page, the user would search for a company to edit, and then the website would display all the results with that company's name.

Before searching:



After searching:



Printing a results table:

```php
if(isset($_POST['search_donation'])){
    $search = $_POST['search'];
    if($search=="all"){
        $sql = "SELECT * FROM Fundraising";
        $stmt = $conn->prepare($sql);
        $result = $stmt->execute();
        echo "<div class='container2'>
                <div id='div_data'>
                    <h2>Results</h2>
                    <table>
                        <tr><td class='labels'>Company</td><td class='labels'>Amount</td><td class
                            ='labels'>Success</td><td class='labels'>Progress</td><td class='labels'
                            >Year</td><td class='labels'>Member Name (First)</td><td class='labels'
                            >(Last)</td></tr>";
        while($row = $stmt->fetch(PDO::FETCH_ASSOC)){
            echo "<tr><td>" . $row['Company'] . "</td><td>" . $row['Amount'] . "</td><td>" .
                $row['Success'] . "</td><td>" . $row['Progress'] . "</td><td>" . $row['Year'] . "
                </td><td>" . $row['Member_First'] . "</td><td>" . $row['Member_Last'] . "</td></tr>"
                ;
        }
        echo "</table></div></div>";
    }
```

One of the benefits of using PHP to print HTML code was being able to use for loops to print things. The Years dropdown menu on the Edit Data page uses a for loop to print all the years, so there wouldn't be many lines of code repeating a similar thing. It also makes it easier for business team members to change in the future because they just have to change one number instead of writing many new lines to add more years.

Using PHP to print HTML for the dropdown menu:

```php
<div>
    <select name='year' id='year' class='textbox'>"
        .(isset($year)?"<option value=".$year.">"
        .$year."</option>":"<option value='-1'>Year
        </option>")." />";
    for($i=2010;$i<=2036;$i++){
        if($year!=$i){
            echo "<option value=".$i.">".$i."</option
          >/>";
        }
    }
    echo "</select>
</div>
```

Only using HTML for the dropdown menu:

```
<div>
    <select name="year" id="year" class="textbox">
        <option value="-1">Year</option>
        <option value="2010">2010</option>
        <option value="2011">2011</option>
        <option value="2012">2012</option>
        <option value="2013">2013</option>
        <option value="2014">2014</option>
        <option value="2015">2015</option>
        <option value="2016">2016</option>
        <option value="2017">2017</option>
        <option value="2018">2018</option>
        <option value="2019">2019</option>
        <option value="2020">2020</option>
        <option value="2021">2021</option>
        <option value="2022">2022</option>
        <option value="2023">2023</option>
        <option value="2024">2024</option>
        <option value="2025">2025</option>
        <option value="2026">2026</option>
        <option value="2027">2027</option>
        <option value="2028">2028</option>
        <option value="2029">2029</option>
        <option value="2030">2030</option>
        <option value="2031">2031</option>
        <option value="2032">2032</option>
        <option value="2033">2033</option>
        <option value="2034">2034</option>
        <option value="2035">2035</option>
        <option value="2036">2036</option>
      </select>
</div>
```

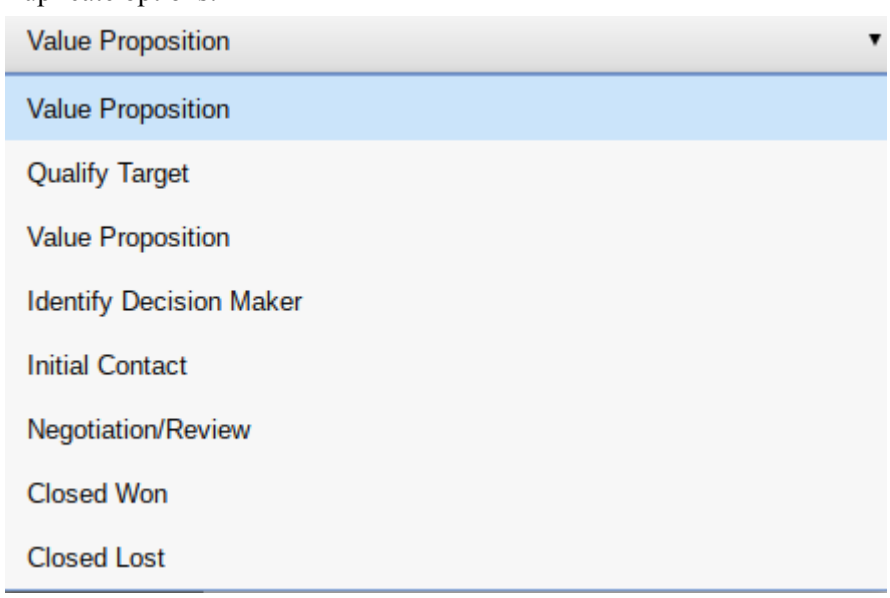
PHP code was also used inside HTML code for the Edit Data form. The PHP code checked if the current information in the data table was null, and if it was, there would be a placeholder in the Edit Data form telling the user what to input. If it wasn't null, the value of the text field would be the current information in the data table. Since if and else statements didn't work in the HTML code, it used ternary operators instead.

```
<div>
    <input type='text' class='textbox' id='success' name='success'".(isset($success)?"value=".$success
        ." placeholder='Success'":"placeholder='Success'")." />
</div>
<div>
    <select name='progress' id='progress' class='textbox'>".(isset($progress)?"<option value="
        .$progress.">".$progress."</option>":"<option value='-1'>Progress</option>")." />"
        .($progress!='Qualify Target'?"<option value='Qualify Target'>Qualify Target</option>":"")."
            />"
        .($progress!='Value Proposition'?"<option value='Value Proposition'>Value Proposition</option
            >":"")."/>"
        .($progress!='Identify Decision Maker'?"<option value='Identify Decision Maker'>Identify
            Decision Maker</option>":"")."/>"
        .($progress!='Initial Contact'?"<option value='Initial Contact'>Initial Contact</option>":""
            )."/>"
        .($progress!='Negotiation/Review'?"<option value='Negotiation/Review'>Negotiation/Review
            </option>":"")."/>"
        .($progress!='Close Won'?"<option value='Closed Won'>Closed Won</option>":"")."/>"
        .($progress!='Close Lost'?"<option value='Closed Lost'>Closed Lost</option>":"")."/>
    </select>
</div>
```

3. Dropdown Menus with Preselected Option

In the Edit Data form, to edit a certain entry, the user would need to see what the current information was and change it. For the textboxes, the value could easily be set to something, but with the dropdown menus, if the value was set to something, it would only preselect the first option in the menu. To preselect the correct option from the current information, there had to be seperate HTML code to set the first option to the correct value. To not have duplicate options and have the correct preselected option, there had to be a way to check if that option was already in the menu.

Duplicate options:



Code for eliminating duplicate options:

```
<div>
    <select name='next' id='next' class='textbox'>".(isset($next)?"<option value=".$next.">".$next."
        </option>":"<option value='-1'>Next Step</option>")." />"
        .($next!='Research Company'?"<option value='Research Company'>Research Company</option>":"")."
            />"
        .($next!='Identify Contact'?"<option value='Identify Contact'>Identify Contact</option>":"")."
            />"
        .($next!='Email Contact'?"<option value='Email Contact'>Email Contact</option>":"")."/>"
        .($next!='Call Contact'?"<option value='Call Contact'>Call Contact</option>":"")."/>"
        .($next!='Visit Contact in Person'?"<option value='Visit Contact in Person'>Visit Contact in
            Person</option>":"")."/>"
        .($next!='Wait for Contact to Respond'?"<option value='Wait for Contact to Respond'>Wait for
            Contact to Respond</option>":"")."/>"
        .($next!='Receive Donation'?"<option value='Receive Donation'>Receive Donation</option>":"")."
            />"
        .($next!='Send Thank You Note'?"<option value='Send Thank You Note'>Send Thank You Note
            </option>":"")."/>"
        .($next!='Update Donor on Team Progress'?"<option value='Update Donor on Team Progress'>Update
            Donor on Team Progress</option>":"")."/>"
        .($next!='Update Donor at End of Season'?"<option value='Update Donor at End of Season'>Update
            Donor at End of Season</option>":"")."/>
    </select>
</div>
```

4. Edit Data Page

This page provides evidence of algorithmic thinking because the user has to select a certain entry and then be able to edit it, and there are lots of things that the code needs to check for it to work.

Pseudo Code	**Explanation**
if *search_data* form is submitted	
- Write the company name from the search form into a text file	This stores the company name for when it will be used later
- Search for the number entries in the Fundraising table that match the search	
- if the number > 0	Checks if there are any results
- Search for the entries in the Fundraising table that match the search	
- Create a table with a form (*edit_data*) for the user to select an entry from	
- *count* = 0	The value of *count* will be used after the *edit_data* form to figure out which entry to select
- while there are results	
- print each row into a table	
- the value of the radio buttons in the form equals *count*	
- *count*++	
- else	
- No results	
if the *edit_data* form is submitted	

- *number* = *count*
- *company* = Read company name from text file
  Reads the name so the code can search for all of the entries again
- Search for all entries in Fundraising where the company name equals *company*
- for (*i*=0; *i*<=*number*; *i*++)
    - *row* = Get data from next row in the search results as an array
      Finds the entry that the user selected in the previous form to get current data from it
    - Set variables equal to data from columns (ex. *amount* = row['Amount'])
    - Store values of the variables in a text file
      Stores current data to be used after the *edit* form is submitted to edit the correct entry
- Create *edit* form
    - Code for the text fields:
      This form displays the current entry and lets the user edit it
        - if the current data in the data table is not null
            - Set the value of that text field to the current data in the table
              This makes sure current data is displayed for the user to edit if it exists
        - else
            - make a placeholder in the text field with the name of what needs to be entered
    - Code for the dropdown menus using *progress* as an example:
        - if the current data (*progress*) in the data table is not null
            - Create the first option where the value is equal to *progress*
              Makes the dropdown menu display current information
            - if the option (*optionName*) != *progress*
              Makes sure there are no duplicate options this part of the code is run for each option
                - Make *optionName* one of the options
        - else
            - Make the first

option = "Progress"
- Make all the normal options

if *edit* form is submitted

- Retrieve values from *edit* form and put them in variables
- Read values (data from before the entry was edited) from text file and store them in variables
- Edit the data

Edits the data which meets one of the success criteria

Makes sure the code only edits the correct entry

5. Reading and Writing in Text Files

Being able to read and write in text files helps meet the success criteria for tracking the progress of a donation since the user would need to edit the data to change the progress. Reading and writing in text files was an appropriate technique because the information from previous HTML forms that were submitted couldn't be saved in a variable to be used after another form was submitted.

After the user submits the search_data form, the company name that they entered into the form gets saved in a text file.

```
if(isset($_POST['search_data'])){
    $search = !empty($_POST['search']) ? trim($_POST['search']) : null;

    $file = fopen("edit_data.txt", "w") or die("Unable to open file!");
    $txt = $search;
    fwrite($file, $txt);
    fclose($file);
```

After the user selects an entry from the search_data results to edit, a text file is opened. Then, the code reads the company name from the text file and searches for all the results again. Then, it will use the row number of the selected row from the edit_data form to get the information from the row that the user selected.

```
if(isset($_POST['edit_data'])){
    $file = fopen("edit_data.txt", "r") or die("Unable to open file!");
    $lines = file("edit_data.txt");
    $search = $lines[0];
    fclose($file);

    $number = $_POST['select']-1;

    $sql = "SELECT * FROM Fundraising WHERE Company = :search";
    $stmt = $conn->prepare($sql);
    $stmt->bindValue(':search', $search);
    $result = $stmt->execute();
```

Then, the information from that row will get saved in a text file to be used later, after the edit form is submitted.

```php
for($i=0; $i<=$number; $i++) {
    $row = $stmt->fetch(PDO::FETCH_ASSOC);
    $company = $row['Company'];
    $amount = $row['Amount'];
    $success = $row['Success'];
    $progress = $row['Progress'];
    $next = $row['Next_Step'];
    $year = $row['Year'];
    $first = $row['Member_First'];
    $last = $row['Member_Last'];

    $file = fopen("edit_data.txt", "w") or die("Unable to open file!");
    $txt = $company . "\n" . $amount . "\n" . $success . "\n" . $progress . "\n" . $next . "\n"
        . $year . "\n" . $first . "\n" . $last;
    fwrite($file, $txt);
    fclose($file);
```

After the user submits the changes, the product has to find the correct entry and change it. The code will read the stored data from the text file and only edit the entry where all of the old data matches it.

```
$file = fopen("edit_data.txt", "r") or die("Unable to open file!");
$lines = file("edit_data.txt");
$c = $lines[0];
$a = $lines[1];
$s = $lines[2];
$p = $lines[3];
$n = $lines[4];
$y = $lines[5];
$f = $lines[6];
$l = $lines[7];
fclose($file);

$companyLength = strlen($c)-1;
$amountLength = strlen($a)-1;
$successLength = strlen($s)-1;
$progressLength = strlen($p)-1;
$nextLength = strlen($n)-1;
$yearLength = strlen($y)-1;
$firstLength = strlen($f)-1;

$c = substr($c, 0, $companyLength);
$a = substr($a, 0, $amountLength);
$s = substr($s, 0, $successLength);
$p = substr($p, 0, $progressLength);
$n = substr($n, 0, $nextLength);
$y = substr($y, 0, $yearLength);
$f = substr($f, 0, $firstLength);

$sql = "UPDATE Fundraising SET Company = :company, Amount = :amount, Success =
    :success, Progress = :progress, Next_Step = :next, Year = :year, Member_First =
    :first, Member_Last = :last WHERE Company = :c AND Amount = :a AND Success = :s
    AND Progress = :p AND Next_Step = :n AND Year = :y AND Member_First = :f AND
    Member_Last = :l";
$stmt = $conn->prepare($sql);
```

6. Download HTML tables as CSV files

A download feature was added since the website didn't have a way to back up data. On the Search page, the user can click a button to download the data. Before downloading the file, the product has to format all the information from the table into CSV format and put it into a CSV file using a PHP array, which has features like a stack.

Formatting and putting the information into an array:

```
while($row = $stmt->fetch(PDO::FETCH_ASSOC)){
    echo "<tr><td>" . $row['Company'] . "</td><td>" . $row['Amount'] . "</td><td>" . $row['Success']
        . "</td><td>" . $row['Progress'] . "</td><td>" . $row['Year'] . "</td><td>" .
        $row['Member_First'] . "</td><td>" . $row['Member_Last'] . "</td></tr>";
    $data = ($row['Company'] . "," . $row['Amount'] . "," . $row['Success'] . "," . $row['Progress']
        . "," . $row['Year'] . "," . $row['Member_First'] . "," . $row['Member_Last'] . "\n");
    array_push($export, $data);
}
```

Downloading the CSV file with some help from knackforge.com (Varthan).

```php
<?php
    header("Content-Disposition: attachment; filename=\"". "export.csv" . "\"");
    readfile ("export.csv");
?>
```

7. Sending reminder emails

The email reminders use PHPMailer to send emails, and a Cron Job, which automatically runs the code for comparing the current day to the reminder date every day (ckhawand). The code checks if a reminder needs to be sent to the assigned member for a certain next step based on days that the client chose, finds the member's email address, and creates and sends an email.

Checking if reminders need to be sent:

```php
//Two day reminder
$current_date = date("Y-m-d");
$current_date = date_create($current_date);
$remind_date = date_add($current_date, date_interval_create_from_date_string('4 days'));
$remind_date = date_format($remind_date, "Y-m-d");
$sql = "SELECT * FROM Fundraising WHERE Due_Date = :date";
$stmt = $conn->prepare($sql);
$stmt->bindValue(':date', $remind_date);
$result = $stmt->execute();
while($row = $stmt->fetch(PDO::FETCH_ASSOC)){
    //Get email address
    $member_assigned_first = $row['Member_First'];
    $member_assigned_last = $row['Member_Last'];
    $next_step = $row['Next_Step'];

    //Check that there is a student assigned and select reminder for next step
    if(!($member_assigned_first == null) && !($member_assigned_last == null) && !($next_step == 'Wait for Contact to Respond') && !
        ($next_step == 'Receive Donation') && !($next_step == 'Update Donor on Team Progress') && !($next_step == 'Update Donor at
        End of Season')){
```

Finding email address:

```php
$sql2 = "SELECT * FROM Students WHERE First = :first AND Last = :last";
$stmt2 = $conn->prepare($sql2);
$stmt2->bindvalue(':first', $member_assigned_first);
$stmt2->bindvalue(':last', $member_assigned_last);
$result2 = $stmt2->execute();
$student = $stmt2->fetch(PDO::FETCH_ASSOC);
$email = $student['Email'];
```

Word Count: 1088

Works Cited

ckhawand. “How to use PHPMailer.” 000webhost forum, https://www.000webhost.com/forum/t/how-to-use-phpmailer/134686?__cf_chl_jschl_tk__=882ea5f5c0b4c9964b6aa27703cdf39324b18d52-1617077943-0-AclixWLzvO825dLrTqaffCRdC2JQ3MjOgmrWpvdWIxR_daTkeyOHjuF6SK8qB7QeKl4AK9OL1vXFHAjlbIRydnVPiuH8zRV6OhXI4PdyUfKRy3k6XyRqI-7R6PWPj47Ql4tPRV9Cpl5AzSF2FriPoghaPRqUKmUv3uibrBc03TUxPSHqZpmwuzWHDaD6L5a8esIOLMdnpQrZQBzdK1n9FbCOq5NrC83FAHcc5QGd1drt8Ogmqlwnha2q0nBC7ie1Oko_hsRPcHxHxOeDd2POx3OQdyMzHPFF6vfPyscbCsaZYHiYbAOx9yqoovg2H2d0f9NZ3OaY2mooguCowGRa07BQaMOSW1uMxBZ9POE468N7NCNMuLGQTG4g9LliHBvqng.

“PHP MySQL Connect to database.” W3Schools, www.w3schools.com/php/php_mysql_connect.asp.

Varthan, Vishnu. “Creating downloadable CSV files in PHP.” KnackForge, https://knackforge.com/blog/vishnu/creating-downloadable-csv-files-php.

Criterion E

Evaluation of the Product

Meeting the Success criteria:

1. Ability to login: The user can log in and create an account.
2. Track the progress of a fundraising opportunity: The database has a column that tracks the progress of the fundraising opportunities.
3. Track basic information of a fundraising opportunity like contact information: There is a database table that stores contact information for fundraising opportunities.
4. Easy to input information and edit information: It is easy for users to input and edit information.
5. Be able to search for different entries: There is a search page that lets the user search for entries.
6. See the progress of any fundraising opportunity: The user can use the search page to see the progress of any fundraising opportunity.
7. See the progress of fundraising opportunities assigned to a specific member: The user can use the search page and dashboard to see the progress of fundraising opportunities assigned to a specific member.
8. See the donation history of a donor: The user can use the search page to see the donation history of a donor.

Feedback from the Client:

Overall, my client really liked the product since it makes it easier for students to track fundraising efforts, he can search by different criteria, it's easy for students to see their own progress, and it's easy to see donation history. One thing he would like to be added is the ability to keep track of the days when students completed certain tasks.

Recommendations for Future Development

One improvement would be adding a way to reset the user's password. This would be beneficial because over the summer, team members might forget their password. Right now, the user would need to create a new account if they forget their password.

Another improvement would be adding a timestamp for when each task is completed. This will help the team keep track of when donors were contacted and make sure the donors are always updated.

To go with the previous recommendation for improvement, I could also create boxes next to the Next Step column on the Dashboard page where the user could check it if they completed the next step. The benefits of this development are that it wouldn't be too much harder to add a timestamp to that action and the user might like that they can check something off when they complete it.

Finally, I could improve it by adding a delete entry. If the user makes a mistake in the fundraising opportunity entry, they can't delete it by deleting all the data in Edit Data fields because the Company column can't be null.

Word Count: 422